CHAPTER 2: DISCOVERY IS ELICITATION, NOT CONVERSATION

Summary: *Discovery fails in a specific and repeatable way: a constraint that was true the whole time surfaces in week six and invalidates the design. Nobody hid it. Nobody asked. This chapter is about the difference between having a conversation with stakeholders and eliciting requirements from them, why the constraints that kill projects are exactly the ones nobody volunteers, and what discovery has to settle before any narrowing is allowed to start.*

Stakeholders Answer Questions. They Do Not Volunteer Constraints.

Somebody tells you what they want built. That sentence contains a solution, and the solution arrived attached to a problem you have not heard yet.

Discovery is the work of telling the **solution somebody named** apart from the **problem they never mentioned**. Every other early activity, the estimate, the prototype, the model pick, the screen designs, spends money or engineering time on an answer whose fit to the real problem is still an open question. Run them in the wrong order and every meeting afterwards is about improving the thing you made rather than about whether it should exist at all.

This needs a method rather than good intentions, and the reason sits in the humans, not in their character. People **respond to what they are asked**, and raise whatever **strikes them as pertinent**. Both halves are entirely reasonable and both halves lose you constraints. A compliance officer does not open with the data residency rule any more than you would open a conversation about a car by mentioning that roads exist. It is background. It is obvious. It is obvious to them.

The Undocumented Assumption, and Two Kinds Nobody Mentions

Name the thing you are hunting: the **undocumented assumption** is a constraint that binds the design, was true before the project started, and has not been said out loud because nobody asked a question that would have surfaced it.

They come in two families, and the two need different questions.

The **obvious-to-them constraint**. Regulatory obligations, data residency, retention rules, an existing contract with a vendor, a union agreement about how work is allocated. These are not secrets. They are so permanent in the stakeholder's world that mentioning them feels like explaining what a week is. Surface them by asking about the category rather than the instance: what rules govern where this data can live, who else has a claim on this process, what has to be true for legal to sign.

The **embarrassing constraint**. The documented process is not the actual process. The data has a quality problem everybody works around. Two teams disagree about who owns the decision and have never resolved it. Nobody volunteers these, because volunteering them is a confession. Surface them by asking about exceptions and workarounds rather than about the process: what happens when the normal path does not work, how often that is, who fixes it.

The **happy-path interview is the named anti-pattern** here. Ask only how the process is supposed to work and you get a tidy requirements document describing something nobody actually does, and the discrepancy then turns up during acceptance, where fixing it costs a redesign instead of a question.

Four Things Discovery Settles Before Anything Narrows

Discovery is finished when four things are agreed, in writing, by people who can commit to them. Not when the meetings stop.

What must be settled	The question that settles it	What goes wrong without it

What must be settled	The question that settles it	What goes wrong without it
The **baseline**	What is the current performance, measured, not remembered?	No way to tell later whether anything improved. Every claim becomes an argument about whether the old process was really that bad.
The **target**	What number, by when, counts as success?	Success is decided retrospectively by whoever is most senior in the room.
The **constraints**	What binds this design regardless of what we would prefer?	The week-six invalidation.
The **owner**	Who is accountable for the outcome moving, and can they commit?	An agreed target nobody is answerable for, which is a target in name only.

All four bind the design. None of them is produced by it. Holding that line is the single most useful discipline in this chapter, because most wrong answers about discovery are build work wearing a discovery costume.

A prompt is not a requirement, it gets drafted and redrafted all the way through the build, measured against criteria that discovery was supposed to hand over, so writing one first hardens guesses that nobody has signed off. A model choice is not a discovery output either. Which model suits the job depends on how accurate, how fast and how cheap the result has to be, and those three numbers are precisely what discovery is still nailing down. Settle the model first and you have resolved a trade-off before anybody stated its terms. Screen designs sit below the value and

feasibility questions in the dependency order, and a whole interface can be redrawn without moving any of those answers an inch.

Two Kinds of Requirement, Separated by the Unit They Are Measured In

Discovery collects requirements of two kinds, and keeping them apart matters because different people sign them off and different tests verify them.

A **functional requirement** names a **behaviour the system owes**. Extract the invoice total. Route the ticket to the right queue. Refuse anything falling outside policy. Every one of those sentences is **about this particular job** and could not be lifted onto a different one without becoming nonsense.

A **non-functional requirement** puts a **bound on how that behaviour is delivered**. Two seconds at the ninety-fifth percentile. Available on an agreed schedule. Under a stated cost per transaction. Records held inside a named jurisdiction. Not one of those four **mentions the work being done**, and that absence is the identifying property: they are **measured in units the subject matter never supplies**.

Which gives you a test taking about four seconds. **Lift the requirement onto a system doing something else entirely** and see whether it survives the move. If it does, it is non-functional, and it belongs among the **binding constraints** rather than in the description of the work.

The reason to bother is that the second kind is where projects die. A functional requirement that was missed produces a feature nobody built, which is embarrassing and fixable. A non-functional requirement that was missed produces a system that does the right thing at the wrong speed, in the wrong place, or at a price nobody approved, and repairing that is usually an architecture change rather than a feature.

Who Is Not in the Room Decides What You Will Not Hear

Structured discovery is mostly a question about attendance. Every category of constraint has an **owner**, and a constraint whose **owner is not in the room** will **not be raised**. That is not a risk to manage. It is arithmetic.

So work the list backwards. Rather than inviting the people the sponsor suggests, enumerate the kinds of constraint that could invalidate this design, then ask who in this organisation is the authority on each. Legal and compliance for what the data may do. Security for who may see it. Operations for what the process actually looks like on a Tuesday. Finance for what the number has to be. The people doing the work today for the exceptions nobody documented.

Then the test that makes it real: for each of those, has **somebody with authority actually said the words**, or are you **assuming**?

When a binding constraint appears late, the defect belongs to the elicitation and not to the client. That framing matters more than it sounds, because every comfortable alternative blocks the repair. Decide the cause was an unusually complicated customer and nothing improves on the next engagement. Decide the cause was one unhelpful stakeholder and nothing improves either, with the added problem that it is not true: people were behaving exactly as described two pages ago, answering what they were asked, and that behaviour is the mechanism that produced the gap rather than an excuse for it.

Words That Hide a Constraint: Just, Only, Simply, Basically

Some words reliably sit on top of unexamined complexity. When somebody says the system just needs to read the ticket and route it, that word is doing load-bearing work: it means the speaker has compressed something they have not examined. The same goes for only, simply and basically.

None of these is a lie. Each is a summary, offered in good faith, by somebody who is not thinking about the part you need. The move is not to challenge the word. It is to ask what happens in the case where it does not hold: *only* the standard tickets, so what happens to the others, how many are there, who handles them now.

The same instinct applies to any noun phrase that names a category without defining it. “Standard requests.” “Normal volume.” “The usual approvals.” Each is a boundary somebody has drawn and not described, and the design has to know where the boundary sits.

How Wrong Answers Are Built on Discovery Questions

Wrong answers here are almost never lazy. They usually name a real, competent, useful activity. What makes them wrong is that the activity belongs somewhere else in the sequence, or rests on an assumption about whose job the discovery is. Three families cover nearly all of them.

- **Producing before agreeing**. The option builds a prototype, drafts a prompt, picks a model, designs screens, quotes a fixed price, or reaches for a shape that worked somewhere else. All of it is genuine work, and all of it ties the engagement to guesses that nobody has checked. Showing people something concrete feels like the fast route to requirements, and it is not, because attention moves instantly to improving what is in front of them and away from whether it was the right thing to make. The test: would choosing this option **spend money, engineering effort or design decisions** on an answer whose **fit nobody has demonstrated**?

- **Treating a binding constraint as a preference**. The option suggests negotiating an exception, escalating for a waiver, or carrying on while somebody sorts it out. Certain obligations hold whatever your timeline says, and carrying on while a waiver is pending simply enlarges the pile of work you throw away if it never arrives. The test: does the requirement come from **outside the organisation** you would be negotiating with?

- **Locating the failure in the room rather than in the questions**. The option reads a late requirement as proof that the customer is complicated or that somebody should have spoken up. Both make it another party's fault, and neither improves the next engagement. The test: could an **earlier question**, put to a **person you could have named in advance**, have brought this out? If it could, the **defect is yours**.

When the scenario says	The answer is usually	The trap is usually

When the scenario says	The answer is usually	The trap is usually
Somebody has described the thing they want built	Pull the request apart into the solution named and the problem behind it, before anything else happens	Any option that begins refining, scoping or pricing what was named
A requirement appeared late and invalidated finished work	Repair the elicitation: which questions get asked, and of whom	Reading it as customer complexity, individual fault, or something to bargain down
An obligation is described as regulatory, jurisdictional or contractual	Accept that it binds, and replan now	Chasing a waiver, or proceeding on the assumption one will arrive
You are asked which activities count as discovery	Only what bounds the design: measured baseline, dated target, binding constraints, accountable owner	Prompt drafts, model selection, screen designs and architecture diagrams, every one of which needs an answer discovery has not delivered yet
A previous engagement resembled this one	Characterise this problem on its own terms, then check whether the resemblance holds up	Opening with the earlier shape, which fixes the engagement to a comparison nobody tested

When the scenario says	The answer is usually	The trap is usually
Production data is offered early	Settle which question the data is meant to answer first	Pulling the data instead of doing discovery, which also starts a privacy discussion nobody came prepared for
The process as described sounds clean	Ask what happens on the days it does not work, and how many of those there are	Writing down the happy path as though it were the process
A requirement has to be classified	Non-functional if it would still read correctly for a system doing entirely different work	Filing a latency, cost, availability or residency number as part of the job description

One habit carries further than the table. When an option describes an **activity you would genuinely do on this project**, ask when you would do it. Most wrong answers in this area are correct answers to a question that comes three weeks later.

What to Remember

- Discovery tells the **named solution** apart from the **unnamed problem**. A request arrives already shaped as an answer, and the problem that answer was meant to solve has **not been said out loud** yet. Anything built before that split rests on a premise nobody checked.

- The **undocumented assumption** is a constraint that **binds the design**, was always true, and was **never stated because nobody asked**. Two kinds: the **obvious-to-them constraint** (regulation, residency, retention, existing contracts) and the **embarrassing constraint** (the documented process is not the real one, the data is bad, ownership is disputed).

- Discovery is complete when **four things are agreed in writing**: a **measured baseline**, a **numeric target with a date**, the **binding constraints**, and a **named owner** accountable for the outcome. Meetings ending is not completion.

- Prompts, model choices, interface designs and architecture diagrams are **build artifacts, not discovery outputs**. Each one presupposes an answer discovery is supposed to produce, so producing them early **freezes assumptions nobody has agreed** to.

- A constraint whose **owner is not in the room** will not be raised. Enumerate the categories of constraint that could invalidate the design, identify who is **authoritative on each**, and check whether they have **actually said the words**.

- When a binding requirement surfaces late, the defect is in the **elicitation, not in the client**. Both comfortable alternatives, that the client is unusual or that a stakeholder should have spoken up, are false and **neither changes the outcome next time**.

- "**Just", "only", "simply" and "basically**" mark **compressed complexity**, as do undefined categories like "standard requests" and "normal volume". Ask what **happens in the case the word excludes**, and how many of those there are.

- A **functional requirement** names a **behaviour this job owes**, such as extracting a total or routing a ticket. A **non-functional requirement bounds how that behaviour is delivered**, in **units the work itself never supplies**: latency, availability, cost per transaction, jurisdiction. The test: **lift it onto a system doing something else**, and if it **still reads correctly** it is non-functional and belongs among the **binding constraints**.